MCP TECHNICAL CHEAT SHEET

Building AI Agents with MCP — Instructor Quick Reference

O'Reilly MCP Bootcamp | github.com/EnkrateiaLucca/mcp-course

CORE CONCEPTS

What is MCP?

The Model Context Protocol (MCP) is an open protocol that standardizes how AI assistants connect with external data sources and tools — a universal adapter that lets AI models talk to any system in a consistent way.

Architecture Components

MCP Hosts

Apps that want data through MCP (Claude Desktop, IDEs); maintain connections to multiple servers

MCP Clients

Protocol clients with 1:1 connections to servers; handle protocol communication

MCP Servers

Lightweight programs exposing tools, resources, prompts, and sampling as separate processes

Transport Layer

`stdio` for parent-child processes, `streamable-http` for remote/networked servers

JSON-RPC MESSAGE FORMAT

MCP uses JSON-RPC 2.0 for all communication.

Request

```
{
  "jsonrpc": "2.0", "id": "req-1",
  "method": "tools/call",
  "params": {
    "name": "get_weather",
    "arguments": {"city": "New
York"}
  }
}
```

Response / Error

```
{
  "jsonrpc": "2.0", "id": "req-1",
  "result": {"content": [
    {"type": "text", "text": "72F,
Sunny"}
  ]}
}
// or: "error": {"code": -32601,
//   "message": "Method not found"}
```

LIFECYCLE

1. **Initialize** — client sends `initialize` with capabilities + clientInfo; server responds with its own capabilities + serverInfo
2. **Initialized** — client sends `notifications/initialized`
3. **Operation** — tool calls, resource/prompt requests, sampling, subscriptions
4. **Shutdown** — client sends `notifications/closed`

THE FOUR CAPABILITIES

Tools (model-controlled)

```
@mcp.tool()
async def calculate(expression:
str) -> str:
    """Evaluate a math
expression"""
    return str(eval(expression))
```

Prompts (user-controlled)

```
@server.list_prompts()
async def list_prompts():
    return
[Prompt(name="debug_code",
        description="Help debug
code")]
```

Resources (application-controlled)

```
@server.list_resources()
async def list_resources():
    return [Resource(uri="file:///
data.json",
                     name="Data
File")]
```

Sampling (server-initiated)

```
result = await
session.create_message(
    messages=[{"role": "user",
               "content": "Analyze
this"}],
    max_tokens=100)
```

ELICITATION

Lets servers request more info from users mid-operation — confirming dangerous actions, filling missing params, clarifying ambiguous requests.

```
response = await
context.request_elicitation(
    f"Delete {len(files)} files
matching "
    f"{file_pattern}?",
options=["yes", "no"])
if response.choice == "yes":
    for f in files: os.remove(f)
```

AUTHORIZATION

MCP supports OAuth 2.0 for secure authentication. Clients handle token acquisition and refresh; servers validate tokens on each request (headers or params).

```
@server.get_auth_requirements()
async def get_auth_requirements():
    return {"type": "oauth2",
        "authorization_url":
"https://.../authorize",
        "token_url": "https://.../
token",
        "scopes": ["read",
"write"]}
```

SECURITY BEST PRACTICES

Input Validation

```
@mcp.tool()
async def read_file(filename: str)
-> str:
    if ".." in filename or
filename.startswith("/"):
        raise ValueError("Invalid
filename")
    safe_path =
os.path.join(ALLOWED_DIR, filename)
    if not
safe_path.startswith(ALLOWED_DIR):
        raise ValueError("Access
denied")
    return open(safe_path).read()
```

Other Practices

CHECKLIST

- Rate-limit tool calls per window
- Least privilege — expose only what's needed
- HTTPS + certificate validation for transport
- Audit-log sensitive tool invocations

UNDER THE HOOD: @MCP.TOOL()

FastMCP registers the function, extracts a JSON Schema from its signature/docstring, routes `tools/call` to it, validates + marshals arguments, and formats the response as `{"content": [...]}`.

COMMON GOTCHAS

Async/Await

All MCP operations are async — don't forget `await`; use `asyncio.run()` for tests

Path Issues

Relative paths break when the server runs from a different cwd — use absolute paths

Resource URIs

Must be valid URIs (`file:///path`), not bare paths

Error Handling

Catch specific errors and return a friendly message; log unexpected ones

TESTING & DEBUGGING

MCP Inspector

```
mcp dev ./my_server.py
# opens browser: tool tester,
resource
# browser, message inspector, live
logs
```

Common Errors

Error	Meaning
Method not found	Server lacks that capability
Invalid params	Check the JSON schema
Transport error	Server crashed / network issue

> **Tip**
>
> Enable verbose logging with `logging.basicConfig(level=logging.DEBUG)` and check Claude Desktop logs at `~/.config/Claude/logs/` (macOS/Linux).

COURSE DEMO PROGRESSION

DEMOS/

00 — Intro to agents
Hand-rolled agent loop, no MCP yet — web search + filesystem tools

01 — Introduction to MCP
Same tools behind a FastMCP server; host/client/server split

02 — Research agent on the SDK
Claude Agent SDK as the MCP host — loop collapses to ~15 lines

03 — Query tabular data
CSV/pandas MCP server driven by the Agent SDK

04 — Production research agent
Intent-grouped tools, HTTP transport, bearer auth, hooks + evals

05 — Link health checker
Agent SDK + dedicated link-checking MCP server

06 — Data analysis agent
FastAPI + SSE, in-process MCP server, deployed to Vercel

07 — Hacks, tips, tools
Ecosystem tools + the `mcp-builder-skill`

08 — MCP security lab
Tool poisoning: attack and defense

09 — Multi-server + subagents
Composes servers the agent didn't write; delegates to a subagent

10 — Sessions
Resume a session or fork it to explore an alternative

COMMON STUDENT QUESTIONS

Why not just use function calling?
MCP is standardized across providers, adds resource management, prompts, and consistent auth/security

Can MCP servers talk to each other?
No — servers are independent; the host/client orchestrates between them

Is MCP only for Claude?
No, it's an open protocol any AI system can implement

How is this different from LangChain?
MCP is a protocol, not a framework — it defines communication, not what you build

TEACHING TIPS

- Start with the "universal adapter" analogy
- Show before/after: AI without MCP vs. with MCP
- Use real examples — weather, files, databases
- Build incrementally, one capability at a time

FURTHER READING

- `modelcontextprotocol.io/specification`
- `modelcontextprotocol.io/docs/learn/architecture`
- `github.com/modelcontextprotocol/servers`